

Introducción

Programación I

UNRN

Universidad Nacional
de **Río Negro**

04

Agosto

Introducción a la Programación

1. En esta clase damos los primeros pasos: qué es un algoritmo, qué es un lenguaje de programación, y cómo se estructura y ejecuta un programa en C.

<https://www.onlinegdb.com/>



Contexto

Creado entre 1969 y 1972 por Dennis Ritchie y Brian Kerningham hace 50 años



Contexto

Influyó en los siguientes lenguajes

C++

C#

Javascript

PHP

Objective-C

Java

Aplicaciones actuales:

Sistemas Operativos y Kernel

Sistemas Embebidos e Internet de las Cosas (IoT)

Motores de Bases de Datos

Desarrollo de Videojuegos y Gráficos



Contenidos de la clase

- 1. Algoritmo y estructuras de control**
Ejemplos de problemas resueltos con algoritmos
- 2. Lenguaje de programación y programación imperativa**
Tipos de datos, constantes, literales y variables
- 3. Asignación de una variable: l-value y r-value**
Código fuente, compilador, linker, ejecutable y ejecución
- 4. Estructura de un programa en C**
Operadores entre tipos primitivos y operadores lógicos

Algoritmo y estructuras de control

- **¿Qué es un algoritmo?**
 - Secuencia finita y ordenada de pasos (instrucciones) que resuelve un problema o realiza una tarea.
- **Características**
 - Finito: siempre tiene que terminar.
 - Preciso: cada paso está bien definido, sin ambigüedad.
 - Tiene entradas (datos que recibe) y salidas (resultados que produce).

Algoritmo y estructuras de control

- Ejemplo: *“indicar los pasos para cocinar milanesas”*

Algoritmo y estructuras de control

- **Ejemplo: “indicar los pasos para cocinar milanesas”**

- a. Juntar los ingredientes: carne, huevo batido y pan rallado.
- b. Tomar el churrasco.
- c. Pasarlo por el huevo batido.
- d. Pasarlo por el pan rallado cuidando que quede bien cubierto.
- e. Poner aceite en una sartén y prender el fuego.
- f. **Si** el aceite está caliente, poner la milanesa. **Si no**, esperar a que caliente.
- g. Dejar cocinar hasta que el lado de abajo esté dorado.
- h. Dar vuelta la milanesa.
- i. Dejar cocinar hasta que el otro lado esté dorado.
- j. Sacar la milanesa y escurrir el aceite.

Finito
Preciso
Entrada/salida

Pseudocódigo (descripción simple y libre de un algoritmo)

```
// 1. Variables
Definir cantidad_churrascos Como Entero
Definir aceite_caliente Como Booleano
Definir lado_A_dorado, lado_B_dorado Como Booleano
```

```
Asignar cantidad_churrascos = 4 // Por ejemplo
Asignar aceite_caliente = Falso
```

```
// 2. EMPANADO
```

```
Mientras cantidad_filetes > 0 Hacer
  Tomar un filete de carne
  Pasar filete por huevo batido
  Pasar filete por pan rallado
  Colocar milanesa armada en una fuente
  cantidad_filetes = cantidad_filetes - 1 //dec
FinMientras
```

```
// 3. Cocción
Prender el fuego de la hornalla
```

```
Mientras haya milanesas crudas en la fuente Hacer
  Si aceite_caliente == Falso Entonces
    Esperar 3 minutos
    aceite_caliente = Verdadero // Cambia el estado
  FinSi
```

```
Poner milanesa en la sartén
lado_A_dorado = Falso
lado_B_dorado = Falso
```

```
// Esperar a que se cocine el primer lado
Mientras lado_A_dorado == Falso Hacer
  Esperar tiempo
  Si mirar_milanesa() == "Dorado" Entonces
    lado_A_dorado = Verdadero
  FinSi
FinMientras
```

```
// Esperar a que se cocine el segundo lado
Mientras lado_B_dorado == Falso Hacer
  Esperar tiempo
  Si mirar_milanesa() == "Dorado" Entonces
    lado_B_dorado = Verdadero
  FinSi
FinMientras
```

```
Sacar milanesa de la sartén
Poner en papel absorbente
FinMientras
```

```
// 4. SALIDA
Servir milanesas con puré o papas fritas.
FinAlgoritmo
```

Algoritmo y estructuras de control

- Ejemplo: *“Escribir el método para determinar si un número es par”*

Algoritmo y estructuras de control

- Ejemplo: *“Escribir el método para determinar si un número es par”*
 - a. Pedirle al usuario que ingrese un número entero.
 - b. Dividir ese número por 2 y calcular el resto (lo que sobra) de la división entera.
 - c. **Si** el resto es igual a cero, significa que el número es par.
 - d. **Si no** (si el resto es uno), significa que el número es impar.
 - e. Informar el resultado al usuario.

Ejemplos de problemas

- **Problemas típicos que resolvemos con algoritmos**
 - Calcular el promedio de varios números.
 - Determinar si un número es primo.
 - Encontrar el máximo o el mínimo de una lista de valores.
 - Convertir una temperatura de Celsius a Fahrenheit.
 - Validar si una contraseña cumple ciertas reglas.
- **En todos los casos**
 - Identificamos los datos de entrada.
 - Definimos los pasos del algoritmo.
 - Obtenemos un resultado (salida).

Algoritmo y estructuras de control

- Los algoritmos en pseudocódigo se estructuran e implementan en lenguajes de programación.
- Los lenguajes definen estructuras de control.
 - Sentencias: las instrucciones se ejecutan una tras otra (asignar un valor a una variable, una operación algebraica, etc).
 - Decisión y selección: if / else, seguir un camino si se cumple una condición.
 - Repetición (iteración): while, for, do-while.
 - Selección de opciones (switch/case).

Lenguaje de programación

- **Definición**

- Conjunto de reglas (sintaxis) y significados (semántica) que permiten escribir algoritmos de forma que una computadora pueda ejecutarlos.

- **Para qué sirve**

- Es el puente entre el pensamiento humano (el algoritmo) y las instrucciones que ejecuta el procesador.
- Ejemplos de lenguajes: C, C++, Python, Java.

- **En esta materia**

- Usamos el lenguaje C para expresar nuestros algoritmos.

Ejemplo en C: nuestro primer programa

miprograma.c



```
#include <stdio.h>

int main(void) {
    printf("Hola, mundo!\n");
    return 0;
}
```

Todo programa en C necesita una función main; printf muestra texto en pantalla.

Ejemplo en C: número par

```
#include <stdio.h>
```

```
int main(void) {
```

```
    int n,r;
```

```
    printf("Ingrese un número: ");
```

```
    scanf("%d", &n);
```

```
    r = n % 2;
```

```
    if (r == 0) {
```

```
        printf("%d es par\n", n);
```

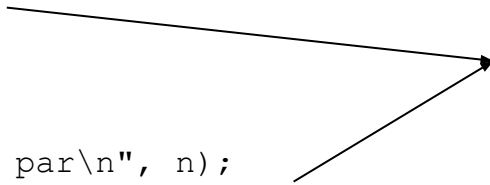
```
    } else {
```

```
        printf("%d es impar\n", n);
```

```
    }
```

```
    return 0;
```

```
}
```



Entrada/salida

Lenguaje de programación

“sistema de comunicación formado por símbolos, palabras y reglas que permite a las personas dar instrucciones a una computadora”

- **Definición**

- Conjunto de reglas (sintaxis) y significados (semántica) que permiten escribir algoritmos de forma que una computadora pueda ejecutarlos.

- **Para qué sirve**

- Es el puente entre el pensamiento humano (el algoritmo) y las instrucciones que ejecuta el procesador.
- Ejemplos de lenguajes: C, C++, Python, Java.

- **En esta materia**

- Usamos el lenguaje C para expresar nuestros algoritmos.

Programación imperativa

“modo de programar que le dice a la computadora el cómo hacer las cosas mediante una lista de pasos, el uso de variables y órdenes directas”

- **Definición**

- Paradigma de programación en el que el programa se describe como una secuencia de instrucciones que van cambiando el estado de la computadora, paso a paso.

- **Idea central**

- Le decimos a la computadora CÓMO hacer las cosas, paso a paso.
- El estado del programa (los valores de las variables) cambia con cada instrucción.

- **C es un lenguaje imperativo**

- Un programa en C es una lista de instrucciones que se ejecutan en orden, salvo que una estructura de control indique lo contrario.

Tipos de datos, constantes, literales y variables

- **Tipo de dato**
 - Categoría que indica qué clase de valores puede tomar un dato y qué operaciones se pueden hacer con él (por ejemplo: int, float, char).
- **Variable**
 - Espacio de memoria con un nombre, cuyo valor puede cambiar durante la ejecución.
- **Constante**
 - Valor que NO cambia durante la ejecución del programa.
- **Literal (*muchas veces se dice “hardcodeado”*)**
 - Valor escrito directamente en el código fuente (por ejemplo: 5, 3.14, 'A', "hola").

Tipos de datos enteros

Tipo (C)	Tamaño	Rango	Ejemplo de Declaración
char	1 byte	-128 a 127	char letra = 'A';
short	2 bytes	-32,768 a 32,767	short contador = 500;
int	4 bytes	-2.14 Mil Millones a +2.14 MM	int saldo = -15000;
unsigned int	4 bytes	0 a 4.29 Mil Millones	unsigned int dist = 4000;
long long	8 bytes	$\pm 9 \times 10^{18}$	long long id = 100000000000;

Los tipos enteros varían en tamaño dependiendo de la arquitectura. Estos son los estándares comunes.

Tipos de datos enteros

- **Calcular el tamaño de una expresión:** `sizeof`
 - La *keyword* - palabra clave - `sizeof(expr)` devuelve el tamaño en bytes de `expr`.

```
#include <stdio.h>
```

```
int main ( void )
```

```
{
```

```
    printf ( "sizeof(char) = %zu \n " , sizeof ( char ));
```

```
    printf ( "sizeof(short) = %zu \n " , sizeof ( short ));
```

```
    printf ( "sizeof(int) = %zu \n " , sizeof ( int ));
```

```
    printf ( "sizeof(long) = %zu \n " , sizeof ( long ));
```

```
    printf ( "sizeof(float) = %zu \n " , sizeof ( float ));
```

```
    printf ( "sizeof(double) = %zu \n " , sizeof ( double ));
```

```
    return (0);
```

```
}
```



```
sizeof(char) = 1
```

```
sizeof(short) = 2
```

```
sizeof(int) = 4
```

```
sizeof(long) = 8
```

```
sizeof(float) = 4
```

```
sizeof(double) = 8
```

Representación de Números Positivos

Conversión Directa a Binario

Los números positivos y sin signo (unsigned) se almacenan en memoria utilizando su representación directa en el sistema binario (base 2).

En los tipos de datos con signo (signed), el bit más significativo (el de más a la izquierda) actúa como bit de signo:

- Si es 0, el número es **positivo**.
- Los bits restantes representan la magnitud del número.

Ejemplo: Representar +5 en 8 bits

1. Valor decimal a convertir:

5

2. Conversión a binario puro:

101

3. Rellenar con ceros (hasta 8 bits):

0000 0101

El bit superior es 0, lo que en un tipo con signo (como char o int) confirma matemáticamente que es un valor positivo.

Representación de Números Negativos

El Complemento a 2

En C, los tipos enteros con signo (signed) utilizan el sistema de **Complemento a 2** para representar números negativos en memoria binaria.

El bit más significativo (el de más a la izquierda) funciona como **bit de signo**:

0 indica un número positivo.

1 indica un número negativo.

Ejemplo: Representar -5 en 8 bits

1. Valor absoluto (5) en binario:

0000 0101

2. Invertir todos los bits (Complemento a 1):

1111 1010

3. Sumar 1 al resultado:

1111 1011

Por lo tanto, en la memoria de la computadora, el número -5 almacenado en un char se ve exactamente como 1111 1011.

Punto Flotante: IEEE 754 “La Notación Científica Digital”

Campos del Formato

Tipos como float (32 bits) y double (64 bits) no se guardan en binario directo. Se dividen en 3 campos:

- **Signo (S):** 1 bit. 0 para positivo, 1 para negativo.
- **Exponente (E):** 8 bits (en float). Define la magnitud desplazando el punto decimal. Se almacena con un "sesgo" (bias de 127).
- **Mantisa (M):** 23 bits (en float). Representa las cifras significativas (fracción) del número, asumiendo un 1. implícito a la izquierda.

Ejemplos y Fórmula

Declaración:

float pi = 3.14159f;

double masa = 5.972e24;

El valor real se calcula uniendo los campos bajo la siguiente fórmula matemática:

$$\text{Valor} = (-1)^S \times (1 + M) \times 2^{E - 127}$$



<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

Punto Flotante: Estándar IEEE 754

<https://www.h-schmidt.net/FloatConverter/IEEE754.html>

Convertidor IEEE 754, 2024-02

	Firmar	Exponente	Mantisa
Valor:	+1	1	1 + 0,5707963705062866
Codificado como:	0	2	
Binario:	☐	128	4788187
	☐	☒ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐ ☐	☒ ☐ ☐ ☒ ☐ ☐ ☒ ☐ ☐ ☐ ☐ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☐ ☒ ☒ ☐ ☒ ☒
Representación decimal		3.141592654	
Valor almacenado en formato flotante:		3.1415927410125732421875	☐ 1
Error debido a la conversión:		0.0000000870125732421875	☐ -1
Representación binaria		01000000010010010000111111011011	
Representación hexadecimal		40490fdb	

Ejemplo en C: tipos de datos y variables

```
#include <stdio.h>

#define PI 3.14159    /* constante simbolica */

int main(void) {
    int edad = 20;    /* variable entera; 20 es un literal */
    float altura = 1.75; /* variable real; 1.75 es un literal */
    char inicial = 'J'; /* variable caracter; 'J' es un literal */
    const int MAX = 100; /* constante entera */

    printf("Edad: %d, Altura: %.2f\n", edad, altura);
    printf("Circunferencia: %.2f\n", 2 * PI * 10);
    return 0;
}
```

Ejemplo en C: Calcular el área de un triángulo

Asignación de una variable: l-value y r-value

- **Asignación (=)**
 - Operación que guarda un valor dentro de una variable, sobrescribiendo el valor anterior.
- **l-value (left value)**
 - Lo que puede aparecer a la izquierda del "=". Representa una ubicación de memoria (una variable).
- **r-value (right value)**
 - Lo que puede aparecer a la derecha del "=". Es un valor: un literal, una variable o una expresión.
- **Regla clave**
 - Un l-value casi siempre puede usarse como r-value, pero un r-value (como un literal) NO puede usarse como l-value.

Ejemplo en C: l-value y r-value

```
int main(void) {  
    int x, y;  
  
    x = 5;          /* x es l-value, 5 es r-value */  
    y = x;          /* y es l-value, x se usa como r-value */  
    x = x + 1;      /* x aparece como l-value y como r-value */  
  
    /* 5 = x;      ERROR: 5 no es un l-value */  
  
    return 0;  
}
```

La línea comentada no compila: no se puede asignar un valor a un literal.

Ejemplo sencillo: entrada y salida

```
#include <stdio.h>

int main(void) {
    int a, b, suma;

    printf("Ingrese el primer numero: ");
    scanf("%d", &a);

    printf("Ingrese el segundo numero: ");
    scanf("%d", &b);

    suma = a + b;    /* asignacion: suma es l-value */

    printf("La suma es: %d\n", suma);

    return 0;
}
```

scanf lee datos de entrada (teclado); printf muestra datos de salida (pantalla).

Proceso de generación del ejecutable

Del código fuente al programa en ejecución

- 1. Código fuente (El Plano de Diseño):** Es el archivo de texto (con extensión `.c`) que escribe el programador. Contiene toda la lógica del programa, pero el procesador de la computadora aún no lo entiende porque está escrito en un lenguaje pensado para humanos.
- 2. Compilador (El Traductor y Revisor):** Actúa como un profesor estricto. Primero, lee el código y detecta cualquier error de sintaxis (;). Si todo está correcto, traduce tus instrucciones a código objeto (lenguaje de máquina) y genera un archivo con extensión `.o` u `.obj`.
- 3. Linker o Enlazador (El Armador Final):** EL archivo objeto está traducido, pero incompleto. Por ejemplo, sabe que si se usó la función `printf`, pero el código real de `printf` está previamente escrito. El Linker busca ese código en las bibliotecas estándar de C y lo "enlaza" o une con el archivo `.o` para completar el rompecabezas.
- 4. Ejecutable (El Producto Terminado):** Es el resultado final del trabajo del Linker. Se genera un archivo binario (como un `.exe` en Windows o un archivo sin extensión en Linux). Es un paquete 100% independiente y autosuficiente.
- 5. Ejecución (La Puesta en Marcha):** Es el momento de la verdad. Cuando se abre el programa, el Sistema Operativo lo carga en la memoria RAM y el procesador comienza a leer y realizar tus instrucciones una por una, rapido.

Ejercicio: suma de valores ingresados

```
#include <stdio.h>

int main(void) {
    int a, b, suma;

    printf("Ingrese el primer numero: ");
    scanf("%d", &a);

    printf("Ingrese el segundo numero: ");
    scanf("%d", &b);

    suma = a + b;    /* asignacion: suma es l-value */

    printf("La suma es: %d\n", suma);

    return 0;
}
```

*scanf lee datos de entrada (teclado);
printf muestra datos de salida (pantalla).*

Estructura de un programa en C

- **Directivas de inclusión**
 - `#include` agrega bibliotecas necesarias (por ejemplo, `stdio.h` para usar `printf` y `scanf`).
- **Variables globales**
 - Se declaran fuera de cualquier función. Son visibles desde cualquier parte del archivo.
- **Función main**
 - Punto de entrada del programa: es lo primero que se ejecuta al correr el programa.
- **Variables locales**
 - Se declaran dentro de `main` (o de otra función). Solo existen mientras se ejecuta esa función.

Ejemplo en C: variables globales y locales

```
#include <stdio.h>    /* Directiva de inclusión */

int contador = 0;     /* variable global: visible en todo el archivo */

int main(void) {      /* función main (entry point).
    int edad = 20;     /* variable local, solo existe en main */

    contador = contador + 1;
    printf("Contador: %d\n", contador);
    printf("Edad: %d\n", edad);

    return 0;
}
```

- *contador es global;*
- *edad solo existe dentro de main.*

Más adelante veremos como se estructura la memoria.

Resumen de la clase

- **Vimos**

- Algoritmos, diagramas de flujo y estructuras de control.
- Qué es un lenguaje de programación y qué es la programación imperativa.
- Tipos de datos, constantes, literales, variables, l-value y r-value.
- El camino del código fuente al ejecutable: compilador y linker.
- La estructura básica de un programa en C (main, variables globales y locales).

Resumen de la clase

- Ejercicios para la próxima:
 - Área del triángulo y el círculo.

Fin

Operadores entre tipos primitivos

- **Aritméticos**

- `+` , `-` , `*` , `/` , `%` (módulo, resto de la división entera).

- **Relacionales**

- `==` , `!=` , `<` , `>` , `<=` , `>=` (comparan valores; devuelven verdadero o falso).

- **De asignación**

- `=` , `+=` , `-=` , `*=` , `/=`.

- **Importante**

- El resultado depende del tipo de los operandos: por ejemplo, la división entre enteros trunca el resultado.

Ejemplo en C: operadores entre tipos primitivos

```
#include <stdio.h>

int main(void) {
    int a = 7, b = 2;
    float c = 7.0, d = 2.0;

    printf("a / b = %d\n", a / b);      /* division entera: 3 */
    printf("a %% b = %d\n", a % b);    /* modulo: 1 */
    printf("c / d = %.2f\n", c / d);   /* division real: 3.50 */
    printf("a == b? %d\n", a == b);    /* 0 (falso) */
    printf("a > b?  %d\n", a > b);     /* 1 (verdadero) */

    return 0;
}
```

Operadores lógicos

- **Y lógico (&&)**
 - Verdadero solo si ambas condiciones son verdaderas.
- **O lógico (||)**
 - Verdadero si al menos una de las condiciones es verdadera.
- **Negación (!)**
 - Invierte el valor lógico de una condición.
- **Uso típico**
 - Combinar varias condiciones dentro de un if o de un while.

Ejemplo en C: operadores lógicos

```
#include <stdio.h>

int main(void) {
    int edad = 20;
    int tieneDni = 1;

    if (edad >= 18 && tieneDni) {
        printf("Puede votar\n");
    }

    if (edad < 12 || edad > 65) {
        printf("Entrada con descuento\n");
    }

    if (!tieneDni) {
        printf("Falta el DNI\n");
    }

    return 0;
}
```

Ejemplo en C: área de un triángulo

```
#include <stdio.h>

int main(void) {
    int base, altura, area;

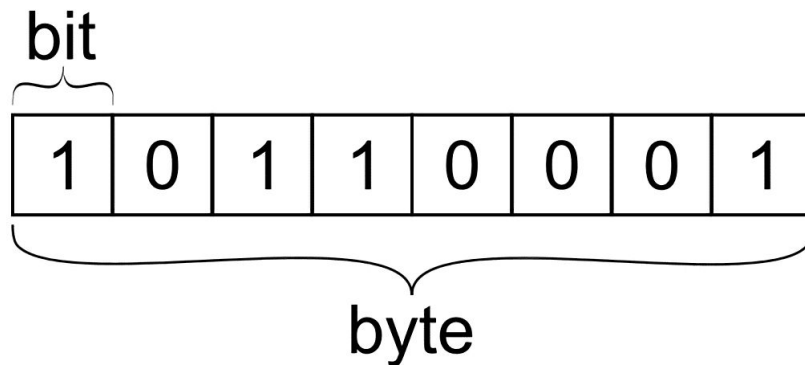
    base = 5;
    altura = 3;
    area = base * altura;    /* el estado cambia paso a paso */
    printf("La base es %d\n", base);
    printf("La altura es %d\n", altura);
    printf("El area es %d\n", area);
    return 0;
}
```

Primero se asignan los valores, luego se calcula el área: cada línea cambia el estado.

Back up Slides

Tipos de datos enteros:

- **char**: Ocupa **1 byte** (8 bits). [256]
- **short**: Ocupa **2 bytes** (16 bits). [65536]
- **int**: Ocupa típicamente **4 bytes** (32 bits) [4294967296]
- **long long**: Ocupa **8 bytes** (64 bits) [1.84467441e19]



$2^8=256$ Posibilidades!

Back up Slides

Modificadores de Formato: Trucos y Variaciones Avanzadas : % [bandera] [ancho] [.precisión] tipo

Modificador	Variación	¿Qué hace exactamente?	Ejemplo de Uso	Salida en Pantalla
%f	%.2f	Redondea y recorta el número	printf("%.2f", 3.14159);	3.14
%d	%5d	Ancho mínimo y Alineación derecha	printf("[%5d]", 42);	[42]
%s	%.3s	Recorte de texto	printf("%.3s", "Programación 1");	Pro
%u	%zu	exclusiva para imprimir el resultado del operador sizeof	printf("%zu", sizeof(int));	4